From PCA to Autoencoders to VAEs

PCA: the linear compression baseline

Given centered data vectors $x \in \mathbb{R}^D$ (mean removed), PCA seeks a k-dimensional representation ($k \ll D$):

$$z = W^\top x, \quad W \in \mathbb{R}^{D \times k}$$

and reconstructs linearly:

$$\hat{x} = Wz = WW^\top x$$

Key idea: choose W to preserve as much variance as possible / minimize reconstruction error.

PCA as an optimization problem (linear reconstruction)

Let $X \in \mathbb{R}^{N \times D}$ be centered data matrix (rows are samples). PCA can be written as:

$$\min_{W \in \mathbb{R}^{D \times k}} \|X - XWW^\top\|_F^2 \quad \text{s.t.} \quad W^\top W = I_k$$

Interpretation:

- $WW^\top$ is an orthogonal projector onto a k-dimensional subspace.
- Best rank-k approximation is given by top-k eigenvectors of covariance.

PCA: maximum variance view

With covariance $\Sigma = \frac{1}{N} X^\top X$,

$$\max_{W^\top W = I_k} \operatorname{tr}(W^\top \Sigma W)$$

Solution: columns of W are eigenvectors of Σ with the k largest eigenvalues.

Limitations of PCA

PCA is powerful but constrained:

- Encoder and decoder are **linear**.
- Learns a single global subspace.
- Cannot easily model complex manifolds (curved structure in data).

Next: **Autoencoders** generalize PCA by learning *nonlinear* maps.

Autoencoder (AE): nonlinear encoder/decoder

An autoencoder learns two functions:

$$z = f_\phi(x) \in \mathbb{R}^k \quad \text{(encoder)}, \qquad \hat{x} = g_\theta(z) \in \mathbb{R}^D \quad \text{(decoder)}$$

Trained by minimizing reconstruction loss:

$$\min_{\phi,\theta} \sum_{n=1}^{N} \mathcal{L}_{rec}\big(x^{(n)}, \hat{x}^{(n)}\big)$$

Common choices:

- Mean squared error: $\|x - \hat{x}\|_2^2$ (Gaussian noise model)
- Binary cross-entropy (BCE) (Bernoulli model), e.g. MNIST in $[0, 1]$

Connection: linear AE $\approx$ PCA

If:

- $f_\phi(x) = W^\top x$ and $g_\theta(z) = Wz$ are **linear**,
- and you use MSE reconstruction loss,
- with appropriate constraints (e.g. orthogonality),

then the solution recovers the PCA subspace (top-k principal components).

Takeaway: PCA is a special case of an autoencoder with linear maps.

Roadmap: from linear factor models to conditional autoencoders

1. No-arbitrage and the stochastic discount factor (SDF)
2. Why linear factor models arise
3. CAPM
4. Fama–French 3-factor and 5-factor models
5. IPCA: characteristics instrument time-varying betas
6. Conditional autoencoder: nonlinear conditional betas

No-arbitrage $\Rightarrow$ stochastic discount factor identity

For any asset with price $P_{i,t}$ and payoff $X_{i,t+1}$,

$$P_{i,t} = \mathbb{E}_t[M_{t+1} X_{i,t+1}] \,.$$

Define the gross return

$$R_{i,t+1} := \frac{X_{i,t+1}}{P_{i,t}}.$$

Then

$$1 = \mathbb{E}_t[M_{t+1} R_{i,t+1}] \,.$$

If $R^e_{i,t+1} := R_{i,t+1} - R_{f,t+1}$ is the *excess* return, then

$$0 = \mathbb{E}_t\left[M_{t+1} R^e_{i,t+1}\right] \,.$$

Why linear factor pricing appears

Start from the excess-return identity:

$$0 = \mathbb{E}_t[M_{t+1} R^e_{i,t+1}] = \mathbb{E}_t[M_{t+1}]\,\mathbb{E}_t[R^e_{i,t+1}] + \mathrm{Cov}_t(M_{t+1}, R^e_{i,t+1}).$$

So

$$\mathbb{E}_t[R^e_{i,t+1}] = -\frac{\mathrm{Cov}_t(M_{t+1}, R^e_{i,t+1})}{\mathbb{E}_t[M_{t+1}]}.$$

If we *restrict* the SDF to be affine in K factors,

$$M_{t+1} = a_t - b_t^\top f_{t+1},$$

then

$$\mathbb{E}_t[R^e_{i,t+1}] = \beta_{i,t}^\top \lambda_t,$$

where

$$\beta_{i,t} := \Sigma_{f,t}^{-1} \mathrm{Cov}_t(f_{t+1}, R^e_{i,t+1}), \qquad \lambda_t := \frac{\Sigma_{f,t} b_t}{\mathbb{E}_t[M_{t+1}]},$$

and $\Sigma_{f,t} = \mathrm{Var}_t(f_{t+1})$.

Cont

Key message:

- No-arbitrage gives the SDF identity.
- A *linear SDF restriction* gives a *linear beta-pricing relation*.
- Linearity is in *expected excess returns and betas*, not in realized returns.

CAPM: one-factor pricing

The CAPM says the only priced factor is the market excess return:

$$R^e_{M,t} := R_{M,t} - R_{f,t}.$$

Expected return relation:

$$\mathbb{E}[R_i] - R_f = \beta_{iM}\big(\mathbb{E}[R_M] - R_f\big),$$

with

$$\beta_{iM} = \frac{\mathrm{Cov}(R_i, R_M)}{\mathrm{Var}(R_M)}.$$

Equivalent time-series regression:

$$R_{i,t} - R_{f,t} = \alpha_i + \beta_{iM}(R_{M,t} - R_{f,t}) + \varepsilon_{i,t}.$$

If CAPM holds, then

$$\alpha_i = 0.$$

CAPM: factor construction and meaning

Factor:

$$\mathsf{MKT}_t := R_{M,t} - R_{f,t}.$$

How it is computed:

- $R_{M,t}$: return on a broad market portfolio
- $R_{f,t}$: one-period risk-free rate

Economic meaning:

- β_{iM} measures exposure to aggregate market risk
- Assets with larger market beta require larger expected excess returns

Limitation:

- Empirically, market beta alone does not fully explain the cross-section of average stock returns

Fama–French 3-factor model

Add two traded factors beyond the market:

$$R_{i,t} - R_{f,t} = \alpha_i + \beta_{iM}(R_{M,t} - R_{f,t}) + \beta_{iS} SMB_t + \beta_{iH} HML_t + \varepsilon_{i,t}.$$

Interpretation:

- $R_M - R_f$: market excess return
- SMB: size factor (*small minus big*)
- HML: value factor (*high book-to-market minus low*)

If the model prices assets exactly, then $\alpha_i = 0$.

How FF3 factors are constructed

Fama–French construct the factors from *2×3 value-weighted sorts*.

Market factor

$$\text{MKT}_t = R_{M,t} - R_{f,t}.$$

Size factor

$$SMB_t = \frac{1}{3}(\text{Small Value} + \text{Small Neutral} + \text{Small Growth}) - \frac{1}{3}(\text{Big Value} + \text{Big Neutral} + \text{Big Growth}).$$

Value factor

$$HML_t = \frac{1}{2}(\text{Small Value} + \text{Big Value}) - \frac{1}{2}(\text{Small Growth} + \text{Big Growth}).$$

Meaning:

- SMB: return spread between small and large firms
- HML: return spread between value and growth firms

Fama–French 5-factor model

The FF5 model adds profitability and investment:

$$R_{i,t} - R_{f,t} = \alpha_i + \beta_{iM}(R_{M,t} - R_{f,t}) + \beta_{iS} SMB_t + \beta_{iH} HML_t + \beta_{iR} RMW_t + \beta_{iC} CMA_t + \varepsilon_{i,t}.$$

New factors:

- RMW: *robust minus weak* profitability
- CMA: *conservative minus aggressive* investment

Motivation:

- Average returns vary with size, value, profitability, and investment

Why go beyond CAPM / FF models?

CAPM and Fama–French models are *linear factor models with fixed loadings*:

$$R^e_{i,t} \approx \beta_i^\top f_t + \varepsilon_{i,t}.$$

But for individual stocks:

- exposures change over time,
- firm characteristics matter,
- the map from characteristics to risk exposure may be nonlinear.

This motivates **conditional factor models**.

IPCA: characteristics are covariances

Let $z_{i,t} \in \mathbb{R}^L$ be observable characteristics for asset i at time t.
General conditional factor model:

$$r_{i,t+1} = \alpha_{i,t} + \beta_{i,t}^\top f_{t+1} + \varepsilon_{i,t+1}.$$

IPCA parameterizes time-varying loadings by characteristics:

$$\beta_{i,t} = z_{i,t}^\top \Gamma_\beta + \nu_{\beta,i,t}, \qquad \alpha_{i,t} = z_{i,t}^\top \Gamma_\alpha + \nu_{\alpha,i,t}.$$

Restricted risk-based version:

$$\Gamma_\alpha = 0 \quad \Rightarrow \quad r_{i,t+1} = z_{i,t}^\top \Gamma_\beta f_{t+1} + \varepsilon_{i,t+1}^*.$$

Interpretation: characteristics instrument for otherwise unobserved, time-varying betas.

IPCA objective

Stack returns and characteristics across assets:

$$r_{t+1} \in \mathbb{R}^N, \qquad Z_t \in \mathbb{R}^{N \times L}.$$

Then the restricted IPCA model is

$$r_{t+1} = Z_t \Gamma_\beta f_{t+1} + \varepsilon^*_{t+1}.$$

The estimation problem is

$$\min_{\Gamma_\beta, \{f_{t+1}\}_{t=1}^{T-1}} \sum_{t=1}^{T-1} \left\| r_{t+1} - Z_t \Gamma_\beta f_{t+1} \right\|_2^2 .$$

What changes relative to FF models?

- factors can be latent,
- loadings vary over time,
- variation in loadings is tied to characteristics.

From IPCA to a conditional autoencoder

IPCA assumes the beta map is *linear* in characteristics:

$$\beta_{i,t} \approx z_{i,t}^\top \Gamma_\beta.$$

Gu–Kelly–Xiu keep the same economic structure,

$$r_{i,t+1} = \beta_{i,t}^\top f_{t+1} + \varepsilon_{i,t+1},$$

but replace the linear beta map with a **nonlinear neural network map**.

Idea:

- left network: characteristics $\to$ conditional betas
- right network: returns $\to$ latent factors
- output: reconstruct each stock return via $\beta_{i,t}^\top f_{t+1}$

Conditional autoencoder: beta network

Input characteristics:

$$z_{i,t}^{(0)} = z_{i,t}.$$

Hidden layers:

$$z_{i,t}^{(\ell)} = g_\ell\Big(b^{(\ell-1)} + W^{(\ell-1)} z_{i,t}^{(\ell-1)}\Big), \qquad \ell = 1, \ldots, L_\beta.$$

Output conditional loading:

$$\beta_{i,t} = b^{(L_\beta)} + W^{(L_\beta)} z_{i,t}^{(L_\beta)}.$$

So instead of

$$\beta_{i,t} = z_{i,t}^\top \Gamma_\beta,$$

the model learns

$$\beta_{i,t} = \beta_\theta(z_{i,t}),$$

a flexible nonlinear function of characteristics.

Conditional autoencoder: factor network

Rather than use the full stock return panel directly, define characteristic-managed returns

$$x_{t+1} = (Z_t^\top Z_t)^{-1} Z_t^\top r_{t+1}.$$

Initialize the factor network with

$$x_{t+1}^{(0)} = x_{t+1}.$$

Hidden layers:

$$x_{t+1}^{(\ell)} = \tilde{g}_\ell\left(\tilde{b}^{(\ell-1)} + \tilde{W}^{(\ell-1)} x_{t+1}^{(\ell)}\right), \qquad \ell = 1, \ldots, L_f.$$

Output latent factors:

$$f_{t+1} = \tilde{b}^{(L_f)} + \tilde{W}^{(L_f)} x_{t+1}^{(L_f)}.$$

If the factor side is one linear layer, the learned factors remain interpretable as portfolios.

Conditional autoencoder: fit and loss

Predicted return for asset i:

$$\hat{r}_{i,t+1} = \beta_{i,t}^\top f_{t+1}.$$

Model-implied reconstruction:

$$\hat{r}_{i,t+1} = \beta_\theta(z_{i,t})^\top g_\psi(x_{t+1}).$$

Training objective:

$$\min_{\theta,\psi} \frac{1}{NT} \sum_{t=1}^{T} \sum_{i=1}^{N} (r_{i,t} - \hat{r}_{i,t})^2 + \lambda \sum_j |\vartheta_j|, \qquad \vartheta := (\theta, \psi).$$

Economic restriction:

- fitted returns must go through a factor structure
- in the no-arbitrage specification, there is no free intercept

Big picture

$$\text{CAPM: } \mathbb{E}[R_i^e] = \beta_{iM}\lambda_M$$

$$\text{FF3/FF5: } R_{i,t}^e = \beta_i^\top f_t + \varepsilon_{i,t}$$

$$\text{IPCA: } r_{i,t+1} = z_{i,t}^\top \Gamma_\beta f_{t+1} + \varepsilon_{i,t+1}^*$$

$$\text{Conditional AE: } r_{i,t+1} = \beta_\theta(z_{i,t})^\top f_{t+1} + \varepsilon_{i,t+1}$$

$$f_{t+1} = g_\psi\Big((Z_t^\top Z_t)^{-1} Z_t^\top r_{t+1}\Big)$$

One-line summary:

- CAPM / FF: fixed linear factor structure
- IPCA: time-varying *linear* betas from characteristics
- Conditional AE: time-varying *nonlinear* betas from characteristics

Autoencoders are not automatically generative

A vanilla AE learns codes $z = f_\phi(x)$ but does not enforce a nice distribution over z.

- Sampling random z may decode to nonsense.
- The latent space can be "holes" and irregular regions.

We want:

- A structured latent distribution (so we can sample),
- A principled objective tied to likelihood.

Next: **Variational Autoencoders (VAEs)**.

VAE: a probabilistic latent-variable model

Assume data is generated by:

$$z \sim p(z) \quad \text{(prior, usually } \mathcal{N}(0, I)\text{)}, \qquad x \sim p_\theta(x \mid z) \quad \text{(decoder / likelihood)}$$

We want to maximize the marginal likelihood:

$$\log p_\theta(x) = \log \int p_\theta(x \mid z) p(z)\, dz$$

Problem: the integral is typically intractable.

Approximate inference: variational posterior

Introduce an approximate posterior (encoder):

$$q_\phi(z \mid x)$$

Common choice (diagonal Gaussian):

$$q_\phi(z \mid x) = \mathcal{N}\big(\mu_\phi(x), \mathrm{diag}(\sigma_\phi(x)^2)\big)$$

Goal: learn θ and ϕ such that:

- $p_\theta(x \mid z)$ generates realistic data,
- $q_\phi(z \mid x)$ approximates the true posterior $p_\theta(z \mid x)$.

Deriving the ELBO (where the objective comes from)

Start from:

$$\log p_\theta(x) = \log \int p_\theta(x, z)\, dz$$

Multiply and divide by $q_\phi(z \mid x)$:

$$\log p_\theta(x) = \log \int q_\phi(z \mid x) \frac{p_\theta(x, z)}{q_\phi(z \mid x)}\, dz = \log \mathbb{E}_{q_\phi(z|x)} \left[\frac{p_\theta(x, z)}{q_\phi(z \mid x)} \right]$$

Apply Jensen's inequality ($\log \mathbb{E}[\cdot] \geq \mathbb{E}[\log(\cdot)]$):

$$\log p_\theta(x) \geq \mathbb{E}_{q_\phi(z|x)} \left[\log p_\theta(x, z) - \log q_\phi(z \mid x) \right]$$

Define the RHS as the **ELBO**: $\mathcal{L}(\theta, \phi; x)$.

ELBO in the familiar form

Since $p_\theta(x, z) = p_\theta(x \mid z)p(z)$:

$$\begin{aligned}\mathcal{L}(\theta, \phi; x) &= \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x \mid z)] + \mathbb{E}_{q_\phi(z|x)}[\log p(z) - \log q_\phi(z \mid x)] \\ &= \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x \mid z)] - \mathrm{KL}\big(q_\phi(z \mid x) \parallel p(z)\big)\end{aligned}$$

Training objective: maximize ELBO, or minimize the negative ELBO:

$$\min_{\theta,\phi} \underbrace{-\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x \mid z)]}_{\text{reconstruction term}} + \underbrace{\mathrm{KL}(q_\phi(z \mid x) \| p(z))}_{\text{regularization}}$$

Why maximizing ELBO is sensible (tightness identity)

There is an exact decomposition:

$$\log p_\theta(x) = \mathcal{L}(\theta, \phi; x) + \mathrm{KL}\big(q_\phi(z \mid x) \parallel p_\theta(z \mid x)\big)$$

Since KL ≥ 0, ELBO is indeed a lower bound:

$$\mathcal{L}(\theta, \phi; x) \leq \log p_\theta(x)$$

Maximizing ELBO does two things:

- increases a lower bound on data likelihood,
- reduces $\mathrm{KL}(q_\phi(z \mid x) \| p_\theta(z \mid x))$ (better inference).

Reparameterization trick (to backprop through sampling)

We need gradients through $z \sim q_\phi(z \mid x)$. For Gaussian q:

$$z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

Now randomness is isolated in ϵ (independent of ϕ), enabling low-variance gradient estimates.

Closed-form KL for diagonal Gaussian posterior

If

$$q_\phi(z \mid x) = \mathcal{N}(\mu, \mathrm{diag}(\sigma^2)), \quad p(z) = \mathcal{N}(0, I),$$

then:

$$\mathrm{KL}(q \| p) = \frac{1}{2} \sum_{i=1}^{k} \left(\mu_i^2 + \sigma_i^2 - \log \sigma_i^2 - 1 \right)$$

So the VAE loss becomes:

$$\mathcal{J}(x) = \underbrace{-\mathbb{E}_{q(z|x)}[\log p_\theta(x \mid z)]}_{\text{BCE for MNIST}} + \underbrace{\frac{1}{2} \sum_i (\mu_i^2 + \sigma_i^2 - \log \sigma_i^2 - 1)}_{\text{KL}}$$

For MNIST: Bernoulli likelihood $\Rightarrow BCE$

If we model pixels (in $[0,1]$) with a Bernoulli likelihood:

$$p_\theta(x \mid z) = \prod_{j=1}^{D} \mathrm{Bernoulli}(x_j;\ \pi_j(z))$$

with decoder outputs $\pi(z) \in (0,1)^D$ (via sigmoid), then:

$$-\log p_\theta(x \mid z) = -\sum_{j=1}^{D} \left[x_j \log \pi_j(z) + (1 - x_j) \log(1 - \pi_j(z))\right]$$

which is exactly the **binary cross-entropy** reconstruction term.

How generation works after training

Once trained:

1. Sample latent $z \sim p(z) = \mathcal{N}(0, I)$
2. Decode: $x \sim p_\theta(x \mid z)$ (or take decoder mean)

Because $q_\phi(z \mid x)$ was regularized toward $p(z)$, random samples from $p(z)$ land in "valid" latent regions.

Summary: PCA $\rightarrow$ AE $\rightarrow$ VAE

- **PCA:** best linear subspace, closed-form eigen-solution.
- **AE:** nonlinear encoder/decoder, minimizes reconstruction error.
- **VAE:** probabilistic latent model, trains by maximizing ELBO:

$$\mathcal{L} = \mathbb{E}_{q(z|x)}[\log p_\theta(x|z)] - \mathrm{KL}(q(z|x)\|p(z))$$

- ELBO derived by Jensen; tightness gap is $\mathrm{KL}(q(z|x)\|p(z|x))$.